

2.3.3 单链表的应用算法设计示例

1. 基于单链表基本操作的算法设计（自学）

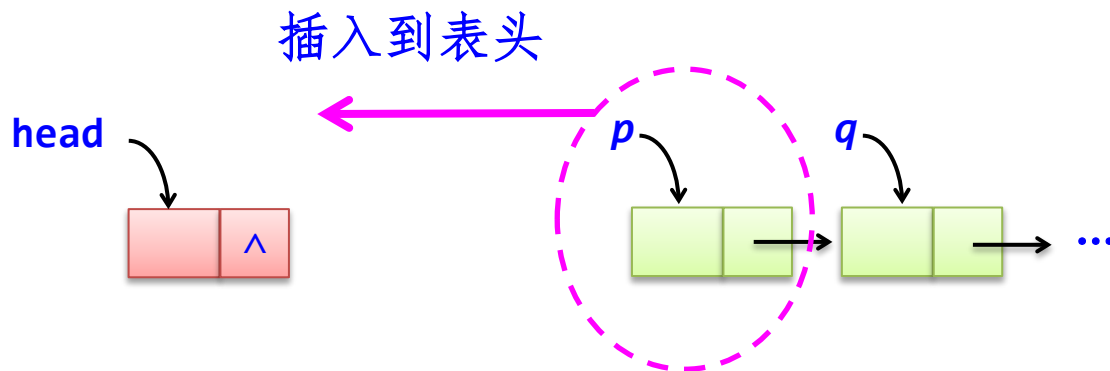
2. 基于整体建立单链表的算法设计

【例2.9】 有一个整数单链表L，设计一个算法逆置L中的所有结点。
例如 $L=(1, 2, 3, 4, 5)$ ，逆置后 $L=(5, 4, 3, 2, 1)$ 。

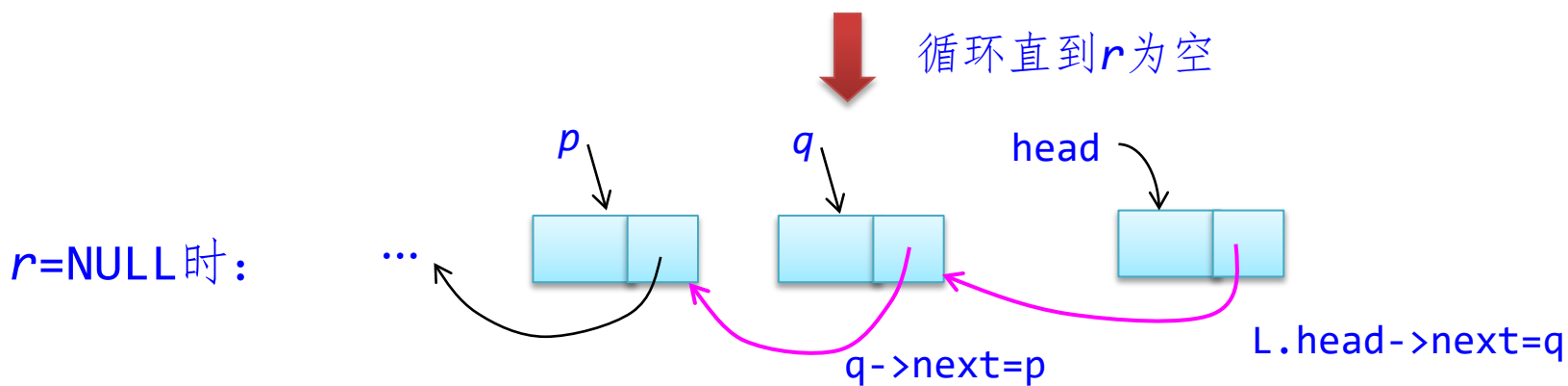
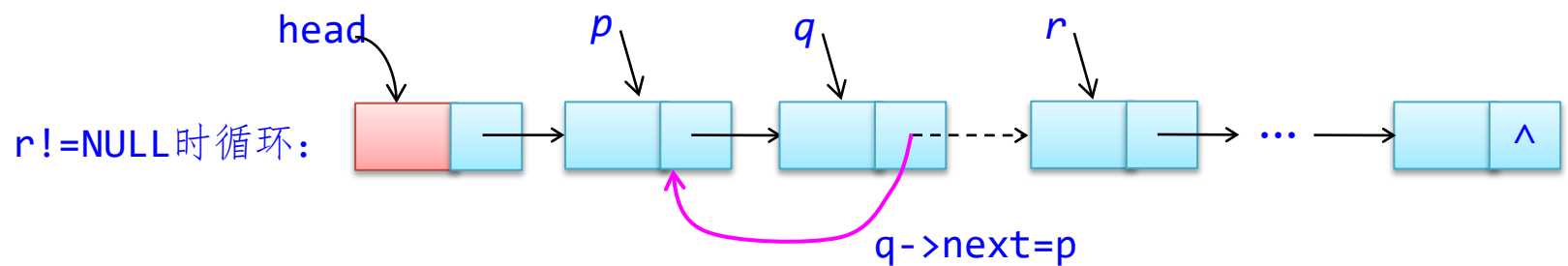
解法1: 采用头插法建表的思路

```
template <typename T>
void Reverse1(LinkList<T>& L)
{  LinkNode<T>* p=L.head->next,*q;
   L.head->next=NULL;
   while (p!=NULL)
   {  q=p->next;
      p->next=L.head->next;
      L.head->next=p;
      p=q;
   }
}
```

//求解算法1
//p指向首结点
//将L置为一个空表
//用p遍历所有数据结点
//q临时保存p结点的后继结点
//将p结点插入到表头

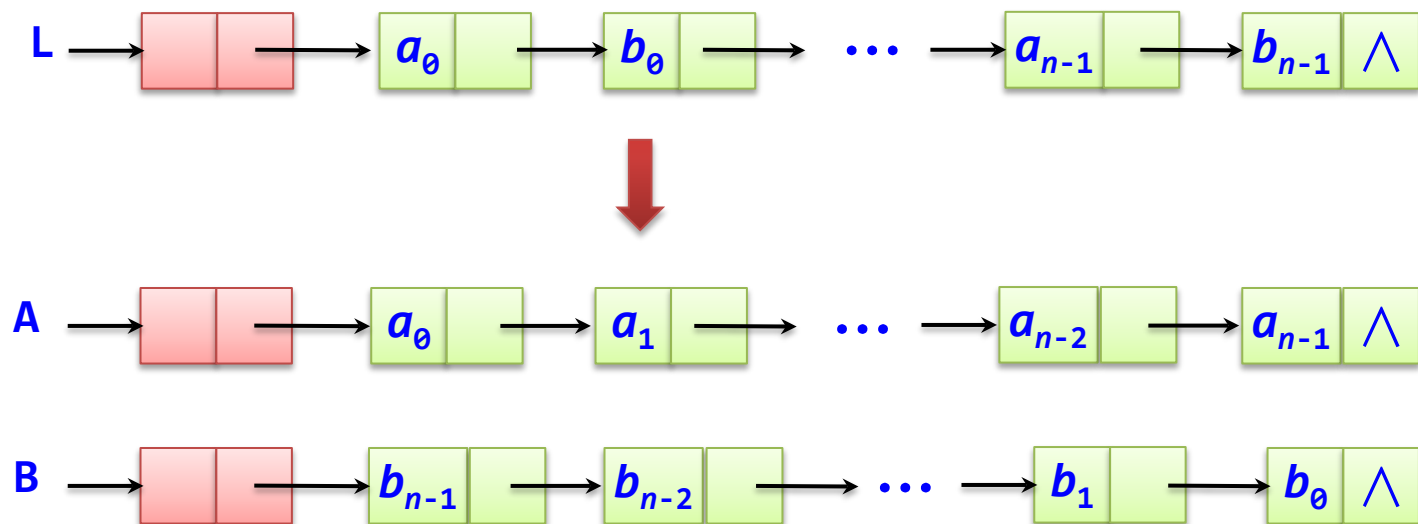


解法2：3指针方法



太复杂，不提倡！

【例2.10】 有一个含 $2n$ 个整数的单链表 $L = (a_0, b_0, a_1, b_1, \dots, a_{n-1}, b_{n-1})$ ，设计一个算法将其拆分成两个带头结点的单链表A和B。
其中 $A = (a_0, a_1, \dots, a_{n-1})$ ， $B = (b_{n-1}, b_{n-2}, \dots, b_0)$ 。



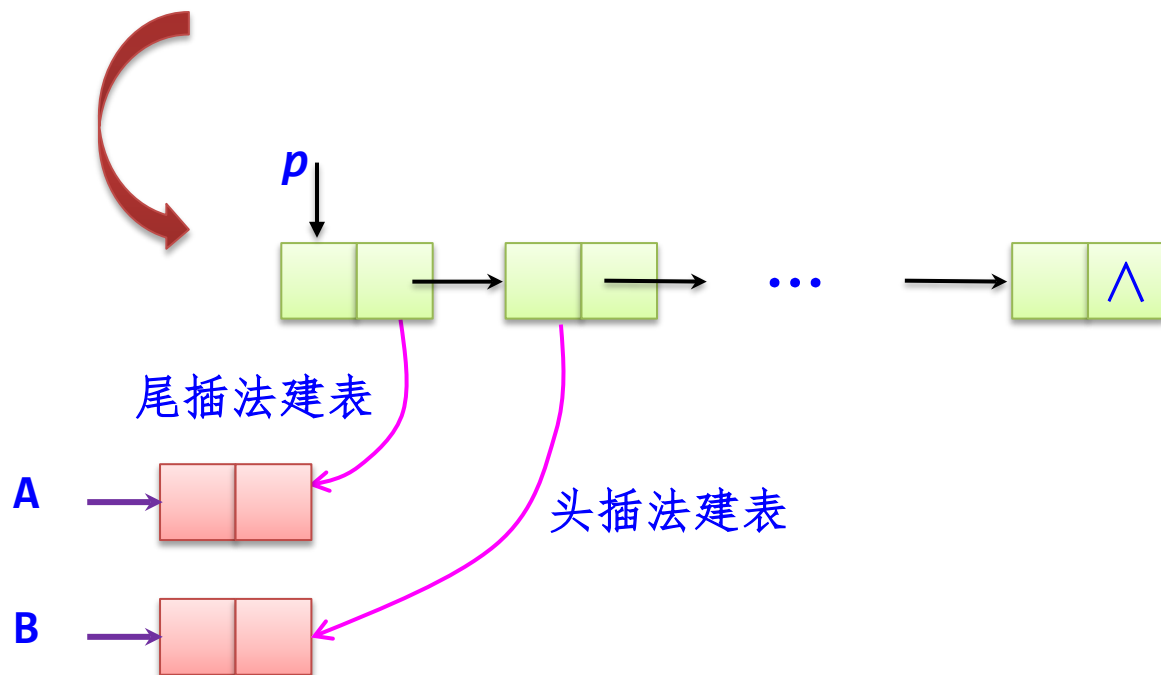


思路

$$L = (a_0, b_0, a_1, b_1, \dots, a_{n-1}, b_{n-1})$$



$$A = (a_0, a_1, \dots, a_{n-1}), \quad B = (b_{n-1}, b_{n-2}, \dots, b_0)$$



```

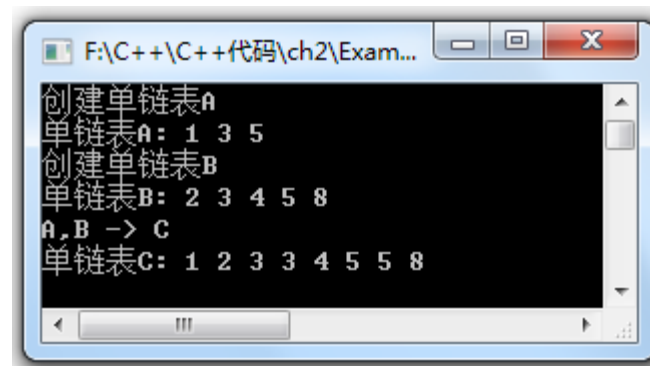
template <typename T>
void Split(LinkList<T>& L, LinkList<T>& A, LinkList<T>& B)
{
    LinkNode<T>* p=L.head->next,*q;    //p指向L的首结点
    LinkNode<T>* r=A.head;            //r始终指向A的尾结点

    while (p!=NULL)                    //遍历L的所有数据结点
    {
        r->next=p;
        r=p;                            //尾插法建立A
        p=p->next;                       //p后移一个结点
        if (p!=NULL)
        {
            q=p->next;                  //临时保存p结点的后继结点
            p->next=B.head->next;        //头插法建立B
            B.head->next=p;
            p=q;                        //p指向q结点
        }
    }
    r->next=NULL;                      //尾结点next置空
}

```

3. 有序单链表的算法设计

【例2.11】有两个递增有序整数单链表A和B，设计一个算法采用二路归并方法将A和B的所有数据结点合并到递增有序单链表C中。要求算法的空间复杂度为 $O(1)$ 。



```
F:\C++\C++代码\ch2\Exam...
创建单链表A
单链表A: 1 3 5
创建单链表B
单链表B: 2 3 4 5 8
A,B -> C
单链表C: 1 2 3 3 4 5 5 8
```

算法的空间复杂度为 $O(1)$?

```

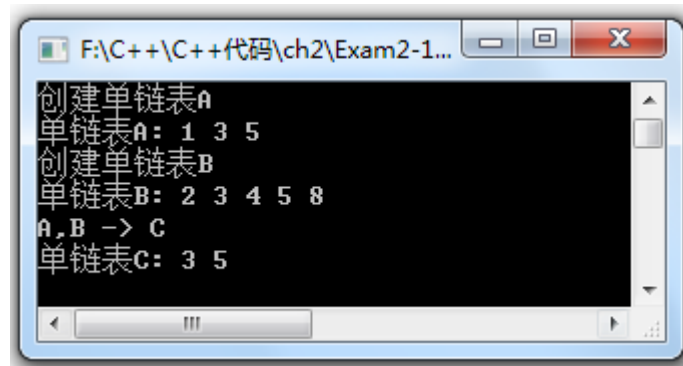
template <typename T>
void Merge2(LinkList<T>& A, LinkList<T>& B, LinkList<T>& C)
{
    LinkNode<T>* p=A.head->next;    //p指向A的首结点
    LinkNode<T>* q=B.head->next;    //q指向B的首结点
    LinkNode<T>* r=C.head;          //r为C的尾结点

    while (p!=NULL && q!=NULL)      //两个单链表都没有遍历完
    {
        if (p->data<q->data)         //将较小结点p链接到C的末尾
        {
            r->next=p;
            r=p;
            p=p->next;
        }
        else                          //将较小结点q链接到C的末尾
        {
            r->next=q;
            r=q;
            q=q->next;
        }
    }

    r->next=NULL;                    //尾结点next置空
    if (p!=NULL) r->next=p;          //将未归并完的结点链接到C的末尾
    if (q!=NULL) r->next=q;
}

```

【例2.12】 有两个递增有序整数单链表A和B，假设每个单链表中没有值相同的结点，但两个单链表中存在相同值的结点，设计一个尽可能高效的算法建立一个新的递增有序整数单链表C，其中包含A和B相同值的结点，要求算法执行后不改变单链表A和B。



```
F:\C++\C++代码\ch2\Exam2-1...
创建单链表A
单链表A: 1 3 5
创建单链表B
单链表B: 2 3 4 5 8
A,B -> C
单链表C: 3 5
```

A=(1,3,5,7,8)

B=(1,2,5,8,10,11)



二路归并



C=(1,5,8)



二路归并 + 尾插法新建单链表C

```

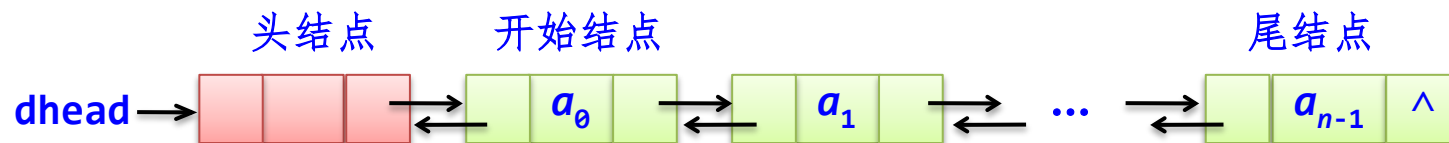
template <typename T>
void Commnodes(LinkList<T>& A, LinkList<T>& B, LinkList<T>& C)
{  LinkNode<T>* p=A.head->next;           //p指向A的首结点
   LinkNode<T>* q=B.head->next;           //q指向B的首结点
   LinkNode<T>* r=C.head;                 //r为C的尾结点

   while (p!=NULL && q!=NULL)             //两个单链表都没有遍历完
   {  if (p->data<q->data)                  //跳过较小的p结点
       p=p->next;
       else if (q->data<p->data)            //跳过较小的q结点
       q=q->next;
       else                                //p结点和q结点值相同
       {  LinkNode<T>* s=new LinkNode<T>(p->data);
          r->next=s;
          r=s;                             //将s结点链接到C的末尾
          p=p->next;
          q=q->next;
       }
   }
   r->next=NULL;                          //尾结点next置空
}

```

- 本算法的时间复杂度为 $O(n+m)$ 。
- 空间复杂度为 $O(\text{MIN}(n, m))$ 。
- 其中 m 、 n 分别为A、B单链表中的数据结点个数，MIN为取最小值函数，因为单链表C中最多只有 $\text{MIN}(n, m)$ 个结点。

2.3.4 双链表



每个结点为**DLinkNode**类型，包括存储元素的列表**data**、存储前驱结点指针**prior**和后继结点指针**next**。

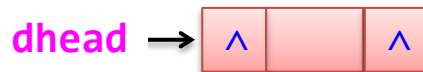
DLinkNode类型



```
template <typename T>
struct DLinkNode                                     //双链表结点类型
{
    T data;                                             //存放数据元素
    DLinkNode<T>* next;                               //指向后继结点的指针
    DLinkNode<T>* prior;                             //指向前驱结点的指针
    DLinkNode():next(NULL),prior(NULL) {}             //构造函数
    DLinkNode(T d):data(d),next(NULL),prior(NULL) {} //重载构造函数
};
```

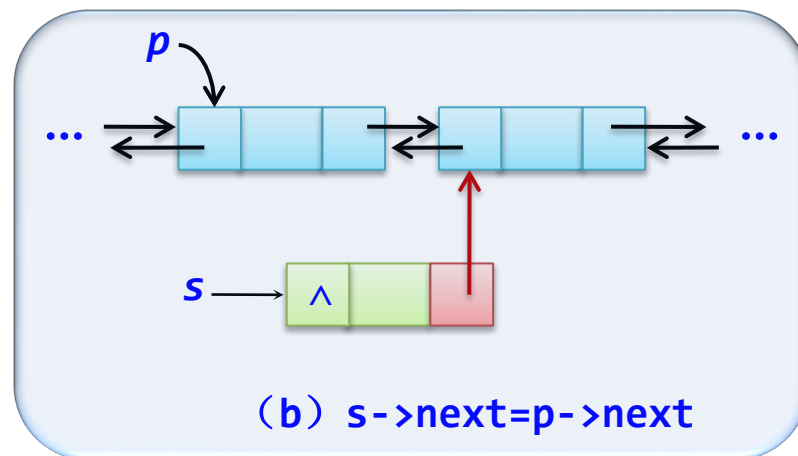
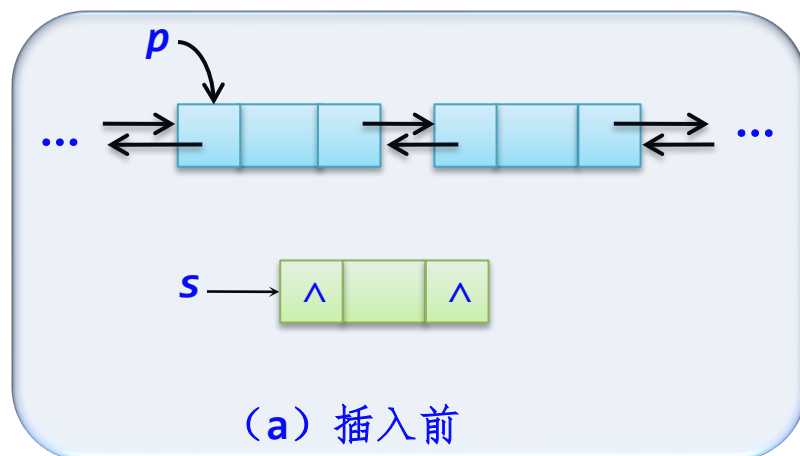
双链表类模板DLinkedList

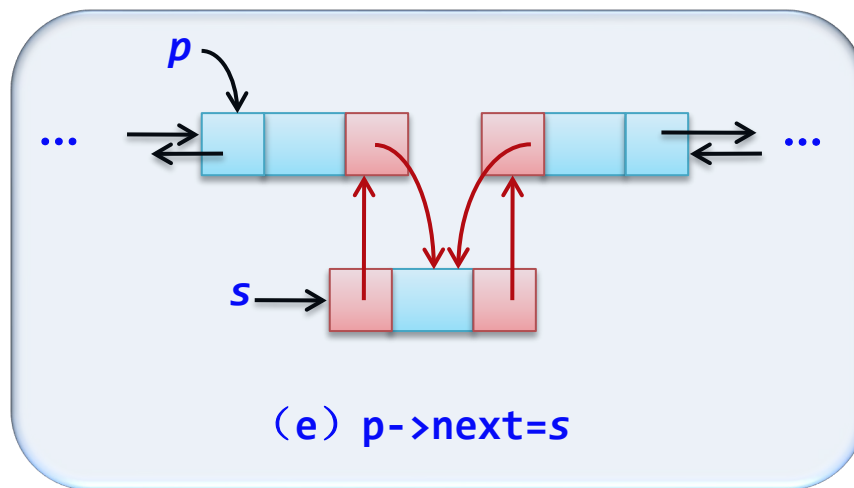
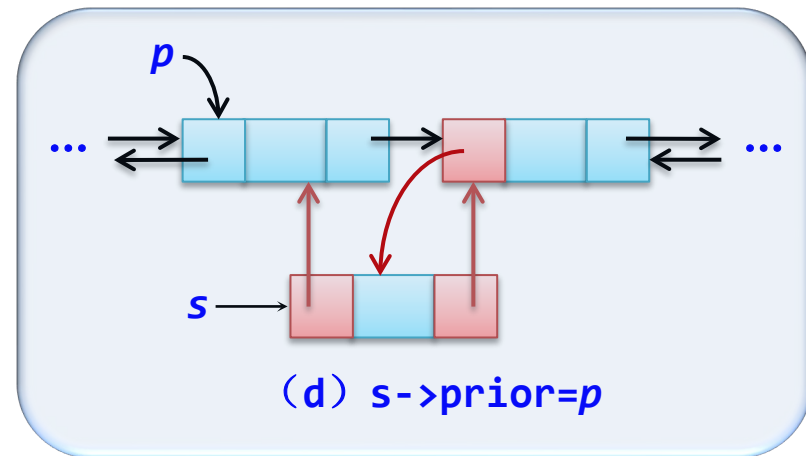
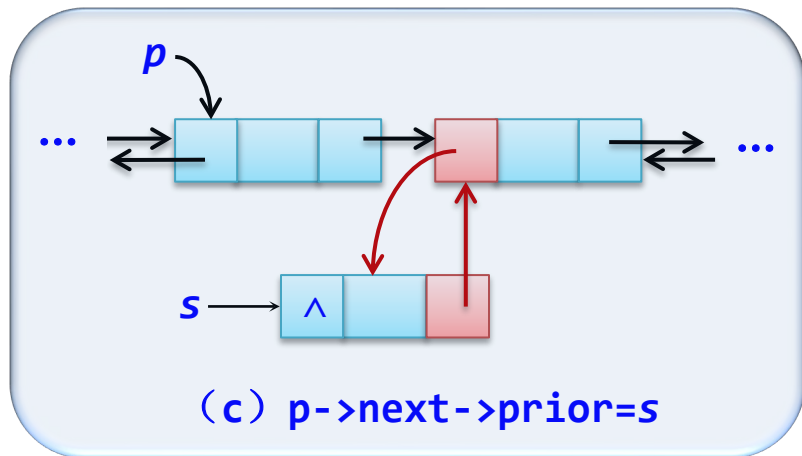
```
template <typename T>
class DLinkedList //双链表类模板
{
public:
    DListNode<T>* dhead; //双链表头结点
    //基本运算算法
};
```



1. 插入和删除结点操作

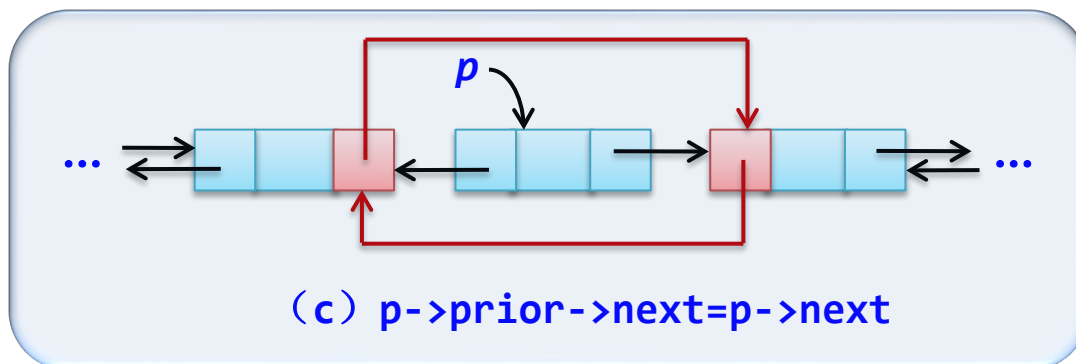
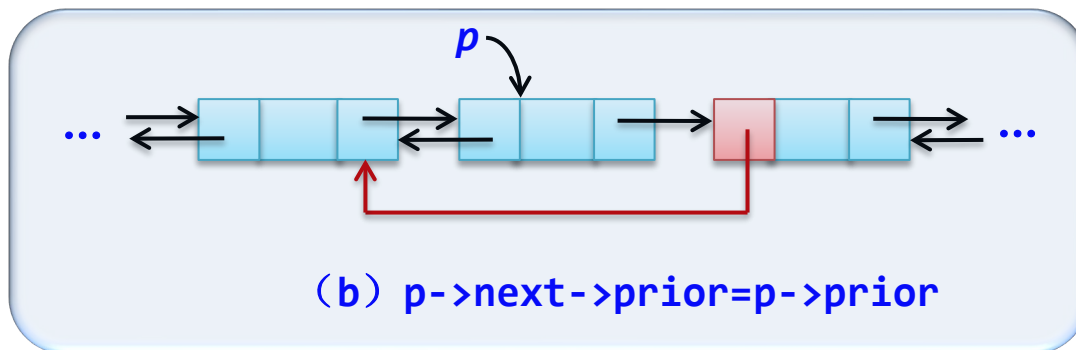
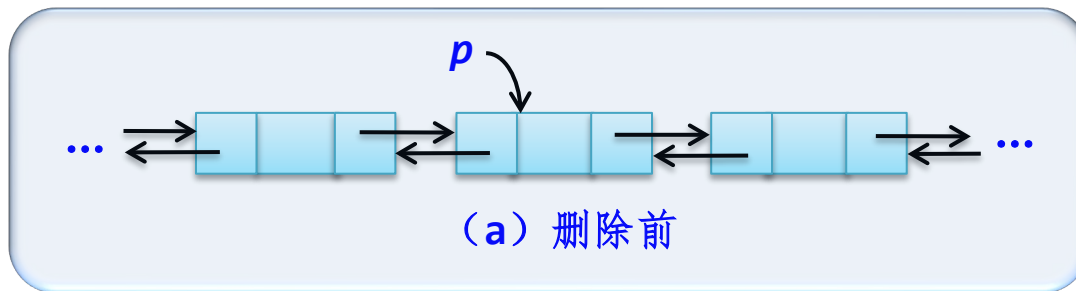
插入结点操作：将结点 s 插入到双链表中 p 结点的后面。





尽可能让间接结点指针修改靠前执行!

删除结点操作：删除双链表中的 p 结点。



2. 整体建立双链表

- 通过一个含有 n 个元素的 a 数组来建立双链表。
- 建立双链表的常用方法有两种：头插法和尾插法。

头插法建表

```
void CreateListF(T a[],int n)
{  DLinkNode<T>* s;
   for (int i=0;i<n;i++)
   {  s=new DLinkNode<T>(a[i]);
      s->next=dhead->next;
      if (dhead->next!=NULL)
         dhead->next->prior=s;
      dhead->next=s;
      s->prior=dhead;
   }
}
```

//头插法建立双链表

//循环建立数据结点

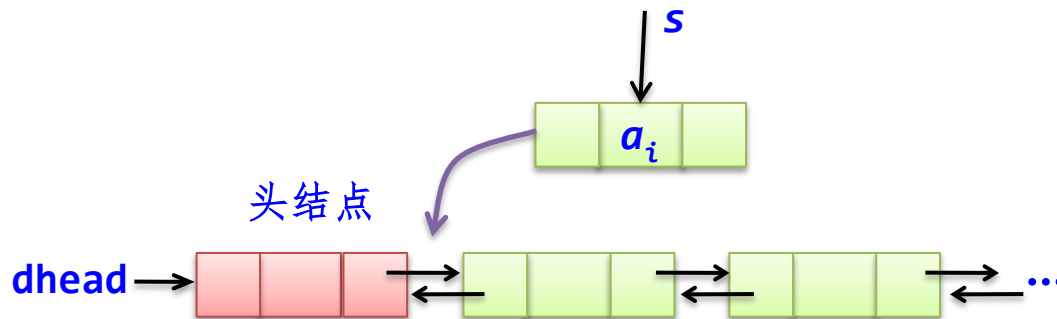
//创建数据结点s

//修改s结点的next成员

//修改头结点的非空后继结点的prior

//修改头结点的next

//修改s结点的prior



尾插法建表

```
void CreateListR(T a[],int n)
{  DLinkNode<T>* s,*r;
   r=dhead;
   for (int i=0;i<n;i++)
   {  s=new DLinkNode<T>(a[i]);
      r->next=s;
      s->prior=r;
      r=s;
   }
   r->next=NULL;
}
```

//尾插法建立双链表

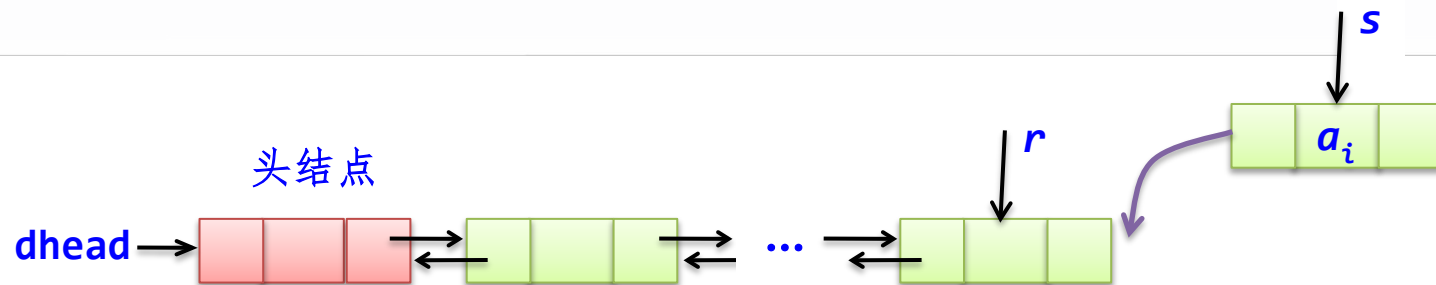
//r始终指向尾结点,开始时指向头结点

//循环建立数据结点

//创建数据结点s

//将s结点插入r结点之后

//将尾结点的next域置为NULL



3. 线性表基本运算在双链表中的实现

- 许多运算算法（如求长度、取元素值和查找元素等）与单链表中相应算法是相同的。
- 涉及结点插入和删除操作的算法需要改为按双链表的方式进行结点插入和删除。

在双链表dhead中序号为i的位置上插入值为e的结点的算法

```
bool Insert(int i,T e)           //在双链表中序号i位置插入值为e的结点
{   if (i<0) return false;       //参数i错误返回false
    DLinkNode<T>* s=new DLinkNode<T>(e); //建立新结点s
    DLinkNode<T>* p=geti(i-1);    //查找序号i-1的结点p
    if (p!=NULL)                 //找到了序号i-1的结点
    {   s->next=p->next;           //修改s结点的next域
        if (p->next!=NULL)        //修改p结点的非空后继结点的prior
            p->next->prior=s;
        p->next=s;                //修改p结点的next域
        s->prior=p;               //修改s结点的prior域
        return true;             //插入成功返回true
    }
    else return false;           //没有找到序号i-1的结点返回false
}
```

说明

也可以在双链表中找到序号为i的结点p（找后继结点），再在p结点之前插入s结点（后继仅仅之前插入新结点）。

在双链表dhead中删除序号为i的结点的算法

```
bool Delete(int i)
{
    if (i < 0) return false;
    DLinkNode<T>* p = geti(i);
    if (p != NULL)
    {
        p->prior->next = p->next;
        if (p->next != NULL)
            p->next->prior = p->prior;
        delete p;
        return true;
    }
    else return false;
}
```

//在双链表中删除序号i位置的结点
//参数i错误返回false
//查找序号i的结点
//找到了序号i的结点p
//修改p结点的前驱结点的next
//修改p结点非空后继结点的prior

//释放空间
//删除成功返回true

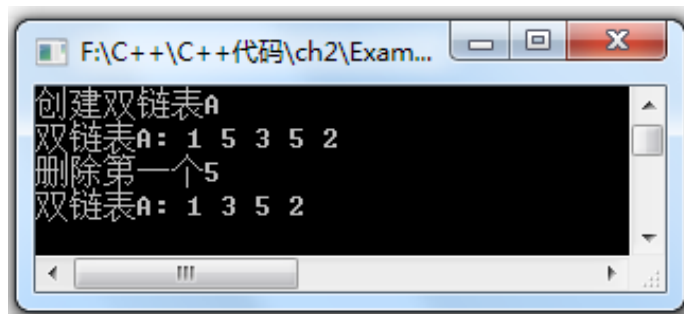
//没有找到序号i-1的结点返回false

说明

也可以找到序号为i-1的结点p（找前驱结点），再删除其后继结点。

2.3.5 双链表的应用算法设计示例

【例2.13】 设计一个算法，删除整数双链表L中第一个值为x的结点，若不存在值为x的结点，则不做任何改变。



```
F:\C++\C++代码\ch2\Exam...
创建双链表A
双链表A: 1 5 3 5 2
删除第一个5
双链表A: 1 3 5 2
```



删除整数双链表L中第一个值为x的结点

```
template <typename T>
void Delx(DLinkedList<T>& L, T x)
{  DLinkNode<T>* p=L.dhead->next;
   while (p!=NULL && p->data!=x)
       p=p->next;

   if (p!=NULL)
   {  p->prior->next=p->next;
      if (p->next!=NULL)
          p->next->prior=p->prior;
      delete p;
   }
}
```

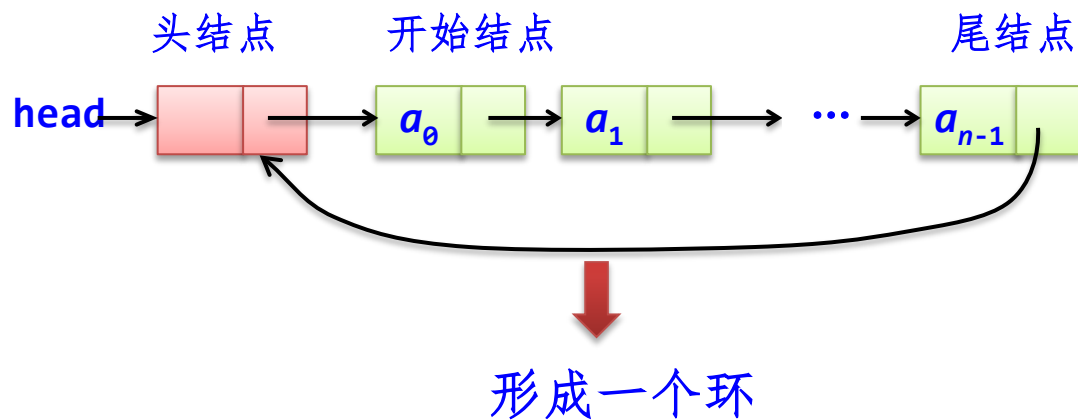
//求解算法
//p指向首结点
//查找第一个值为x的结点

//找到值为x的结点p
//删除p结点

//释放空间

2.3.6 循环链表

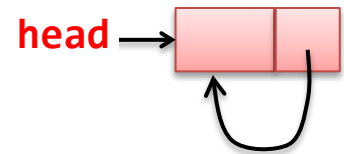
1. 循环单链表



循环单链表类模板CLinkList

```
template <typename T>
class CLinkedList                                //循环单链表类
{
public:
    LinkNode<T>* head;                          //循环单链表头结点
    CLinkedList()                               //构造函数,创建空循环单链表
    { head=new LinkNode<T>();                  //构成循环的空链表
      head->next=head;
    }
    ~CLinkList()                                //析构函数,销毁循环单链表
    { LinkNode<T>* pre,*p;
      pre=head;p=pre->next;
      while (p!=head)                          //用p遍历并释放其前驱结点
      { delete pre;                             //释放pre结点
        pre=p;p=p->next;                       //pre,p同步后移一个结点
      }
      delete pre;                              //释放尾结点
    }
    //线性表的基本运算算法
};
```

结点类型
与非循环
单链表中的
结点类型
相同



循环单链表的插入和删除结点操作与非循环单链表的相同，所以两者的许多基本运算算法是相似的，主要区别如下：

- 初始只有头结点`head`，在循环单链表的构造方法中需要通过`head->next=head`语句置为空表。
- 循环单链表中涉及查找操作时需要修改表尾判断的条件，例如，用`p`遍历时，尾结点满足的条件是`p->next==head`而不是`p->next==NULL`。

【例2.16】编写一个程序求解约瑟夫（Joseph）问题。有 n 个小孩围成一圈，给他们从1开始依次编号，从编号为1的小孩开始报数，数到第 m ($0 < m < n$) 个小孩出列，然后从出列的下一个小孩重新开始报数，数到第 m 个小孩又出列， $\dots$ ，如此反复直到所有的小孩全部出列为止，求整个出列序列。

如当 $n=6$ ， $m=5$ 时的出列序列是5，4，6，2，3，1。

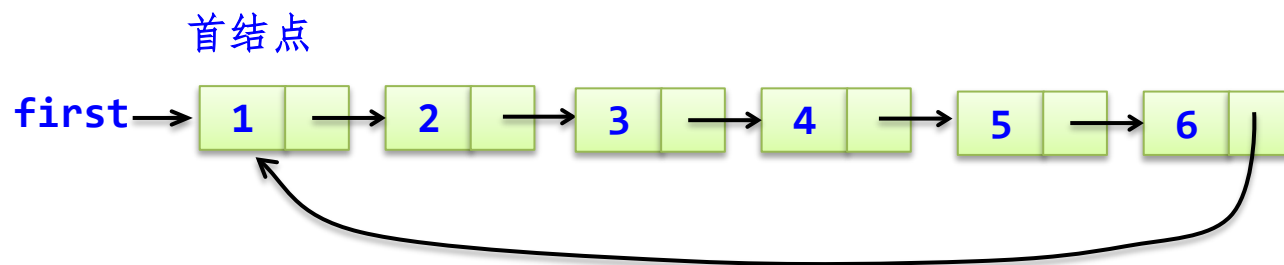


(1) 设计存储结构

本题采用不带头结点的循环单链表存放小孩圈，其结点类如下：

```
struct Child                                //小孩结点类型
{  int no;                                //小孩编号
   Child* next;                            //下一个结点指针
   Child(int d):no(d),next(NULL) {}      //构造函数
};
```

例如， $n=6$ 时的初始循环单链表如下图所示，**first**指向开始报数的小孩结点，初始时指向首结点。



(2) 设计基本运算算法

设计一个求解约瑟夫问题的Joseph类，其中包含：

- n 、 m 整型成员和首结点指针**first**成员。
- **CreateList**函数用于建立有 n 个结点的不带头结点的循环单链表**first**。
- **Jsequence**函数用于产生约瑟夫序列。

class Joseph	//求解约瑟夫问题类
{ int n,m;	
Child* first;	//小孩循环单链表的首结点
public:	
Joseph(int n1,int m1):n(n1),m(m1) {}	//构造函数
void CreateList()	//创建小孩循环单链表
{ first=new Child(1);	//循环单链表首结点
Child* r=first,*p;	//r为尾结点指针
for (int i=2;i<=n;i++)	
{ p=new Child(i);	//建立一个编号为i的新结点p
r->next=p;	//将p结点链到末尾
r=p;	
}	
r->next=first;	//构成首结点为first的循环单链表
}	

```

void Jsequence()                //输出约瑟夫序列
{
    Child* p,*q;
    for (int i=1;i<=n;i++) //共出列n个小孩
    {
        p=first;
        int j=1;
        while (j!=m)        //从first结点开始报数，报到第m个结点
        {
            j++;              //报数递增
            p=p->next;        //移到下一个结点
        }
        cout << p->no << " "; //该结点的小孩出列
        q=p->next;            //q指向结点p的后继结点
        p->no=q->no;          //将结点q的值复制到结点p
        p->next=q->next;      //删除q结点
        delete q;            //释放结点空间
        first=p;             //从结点p重新开始
    }
    cout << endl;
}
};

```

(3) 设计主程序

设计如下主程序求解一个约瑟夫序列。

```
int main()
{   int n=6,m=3;
    Joseph L(n,m);
    cout << "n=" << n << ", m=" << m << "的约瑟夫序列:" << endl;
    L.CreateList();
    L.Jsequence();
    return 0;
}
```

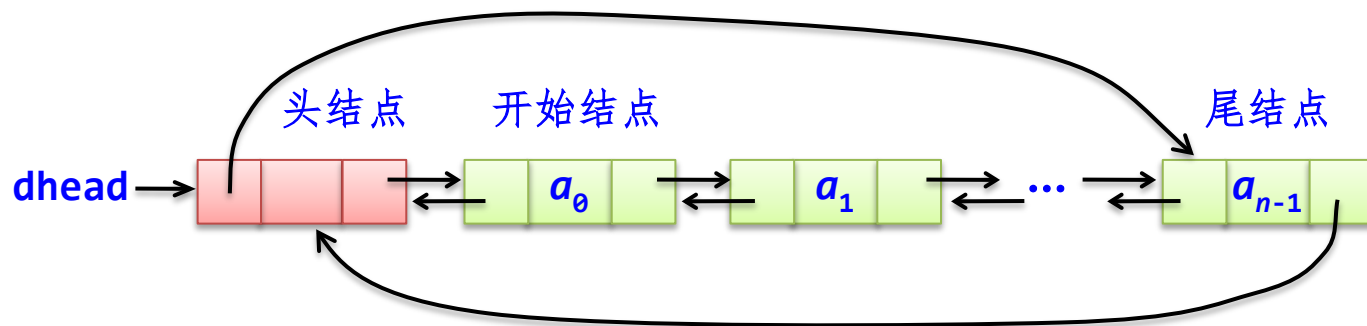
(4) 执行结果

本程序的执行结果如下：

n=6, m=3的约瑟夫序列：

3 6 4 2 5 1

2. 循环双链表



- 形成两个环
- 可以快速找到尾结点

循环双链表类模板DCLinkList

```
template <typename T>
class CDLinkList
{
public:
    DLinkNode<T>* dhead;
    CDLinkList()
    {
        dhead=new DLinkNode<T>();
        dhead->next=dhead;
        dhead->prior=dhead;
    }
    ~CDLinkList()
    {
        DLinkNode<T>* pre,*p;
        pre=dhead;p=pre->next;
        while (p!=dhead)
        {
            delete pre;
            pre=p;p=p->next;
        }
        delete pre;
    }
    //基本运算算法
};
```

//循环双链表类模板

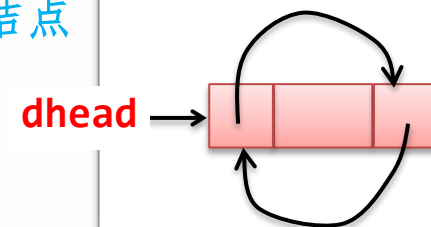
//循环双链表头结点
//构造函数,创建空表
//构成循环的空链表

//析构函数,销毁

//p遍历并释放前驱结点
//释放pre结点
//pre,p同步后移一个结点

//此时释放尾结点

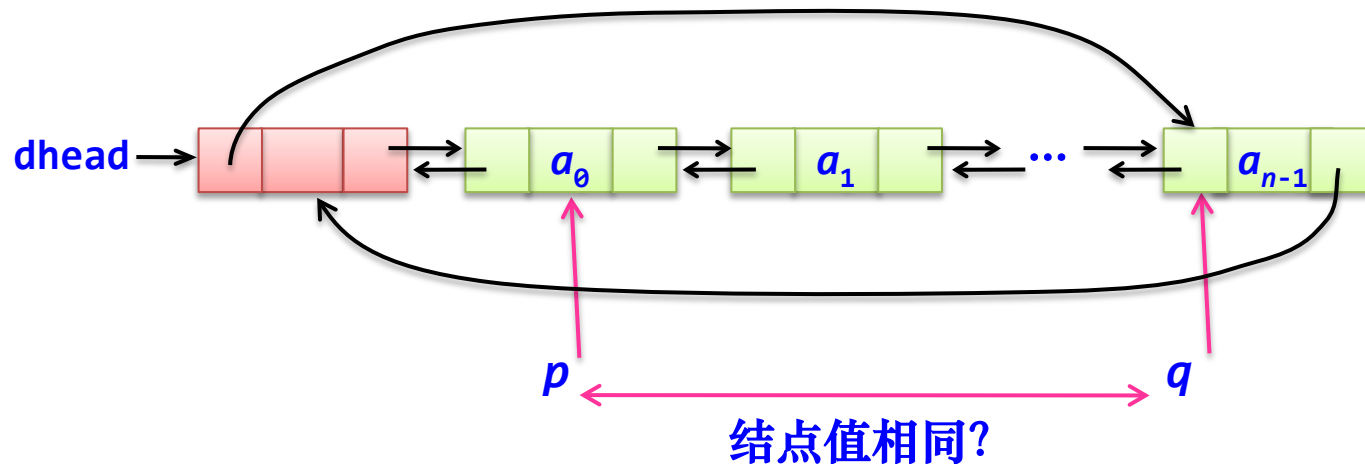
结点类型
与非循环
双链表中的
结点类型
相同



循环双链表的插入和删除结点操作与非循环双链表的相同，所以两者的许多基本运算算法是相似的，主要区别如下：

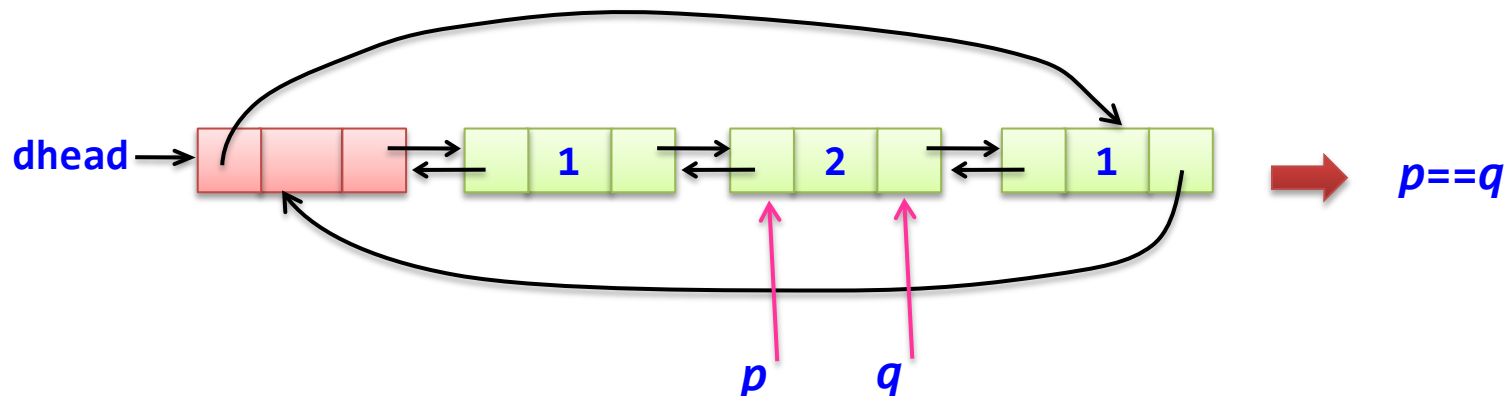
- 初始只有头结点`dhead`，在循环双链表的构造方法中需要通过`dhead->prior=dhead`和`dhead->next=dhead`两个语句置为空表。
- 循环双链表中涉及查找操作时需要修改表尾判断的条件，例如，用`p`遍历时，尾结点满足的条件是`p->next==dhead`而不是`p->next==NULL`。

【例2.18】有一个带头结点的循环双链表L，其结点data成员值为整数，设计一个算法，判断其所有元素是否对称。如果从前向后读和从后向前读得到的数据序列相同，表示是对称的；否则不是对称的。

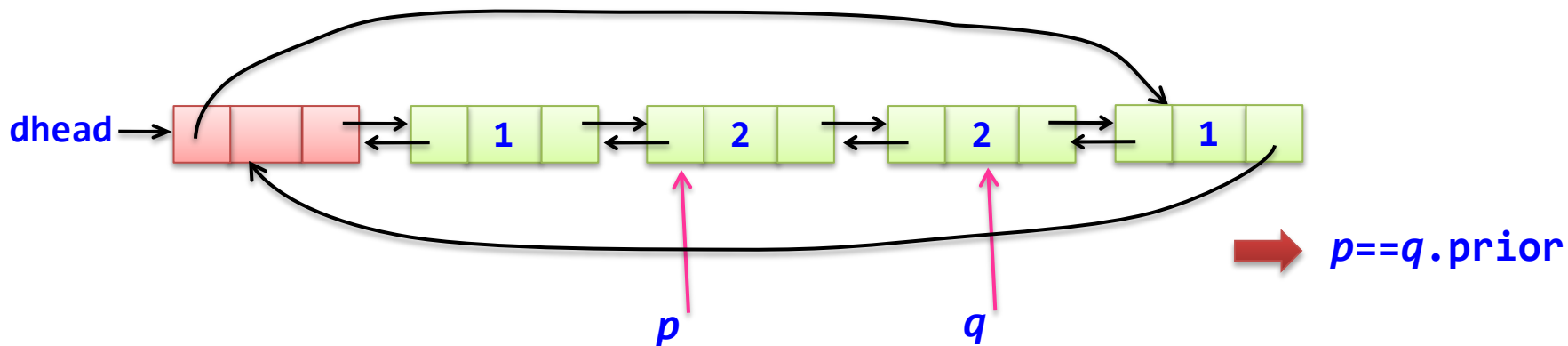


循环结束条件?

(1) 结点个数为奇数



(2) 结点个数为偶数



template <typename T>	
bool Symm (CDLinkedList<T>& L)	//求解算法
{ bool flag=true;	//flag表示L是否对称, 初始时为真
DListNode<T>* p=L.dhead->next;	//p指向首结点
DListNode<T>* q=L.dhead->prior;	//q指向尾结点
while (flag)	
{ if (p->data!=q->data)	//对应结点值不相同, 置flag为假
flag=false;	
else	
{ if (p==q p==q->prior)	//满足结束条件退出循环
break;	
q=q->prior;	//q前移一个结点
p=p->next;	//p后移一个结点
}	
}	
return flag;	
}	

2.4 顺序表和链表的比较

1. 基于空间的考虑

$$\text{存储密度} = \frac{\text{结点中数据本身占用的存储量}}{\text{整个结点占用的存储量}}$$

一般地，存储密度越大，存储空间的利用率就越高。

- 顺序表的存储密度为1，而链表的存储密度小于1。仅从存储密度看，顺序表的存储空间利用率高。
- 顺序表需要预先分配初始空间，所有数据占用一整片地址连续的内存空间，如果分配的空间过小，易出现上溢出，需要扩展空间导致大量元素移动而降低效率；如果分配的空间过大，会导致空间空闲而浪费。而链表的存储空间是动态分配的，只要内存有空闲，就不会出现上溢出。
- **结论：**当线性表的长度变化不大，易于事先确定的情况下，为了节省存储空间，宜采用顺序表作为存储结构。当线性表的长度变化较大，难以估计其存储大小时，为了节省存储空间，宜采用链表作为存储结构。

2. 基于时间的考虑

- 顺序表具有随机存取特性，给定序号查找对应的元素值的时间为 $O(1)$ ，而链表不具有随机存取特性，只能顺序访问，给定序号查找对应的元素值的时间为 $O(n)$ 。
- 在顺序表中插入和删除操作时，通常需要平均移动半个表的元素。而在链表中插入和删除操作仅仅需要修改相关结点的指针成员，不必移动结点。
- **结论：**若线性表的运算主要是查找，很少做插入和删除操作，宜采用顺序表作为存储结构。若频繁地做插入和删除操作，宜采用链表作为存储结构。

2.5 线性表的应用-两个多项式相加

2.5.1 问题描述

多项式 $p(x) = c_1x^{e_1} + c_2x^{e_2} + \dots + c_mx^{e_m}$



求两个多项式相加的程序

例如, $p(x)=2x^3+3.2x^5-6x+10$, $q(x)=6x+1.8x^5-2x^3+x^2-2.5x^4-5$



$$r(x)=p(x)+q(x)$$

$$r(x)=5x^5-2.5x^4+x^2+5$$

两个多项式的数据分别存放在abc1.in和abc2.in文本文件中，要求相加的结果多项式的数据存放在abc.out文本文件中。

abc1.in文件

```
4
2 3
3.2 5
-6 1
10 0
```

$$p(x)=2x^3+3.2x^5-6x+10$$

abc2.in文件

```
6
6 1
1.8 5
-2 3
1 2
-2.5 4
```

$$q(x)=6x+1.8x^5-2x^3+x^2-2.5x^4-5$$

abc.out文件



```
第1个多项式:  [[2.0, 3],[3.2, 5],[-6.0, 1],[10.0, 0]]
排序后结果:    [[3.2, 5],[2.0, 3],[-6.0, 1],[10.0, 0]]
第2个多项式:  [[6.0, 1],[1.8, 5],[-2.0, 3],[1.0, 2],[-2.5, 4],[-5.0, 0]]
排序后结果:    [[1.8, 5],[-2.5, 4],[-2.0, 3],[1.0, 2],[6.0, 1],[-5.0, 0]]
相加多项式:    [[5.0, 5],[-2.5, 4],[1.0, 2],[5.0, 0]]
```

$$p(x) = c_1 x^{e_1} + c_2 x^{e_2} + \cdots + c_m x^{e_m}$$



(c_1, e_1) (c_2, e_2) $\cdots$ (c_m, e_m)  多项式线性表



多项式项

ADT PolyClass

//多项式抽象数据类型

{ 数据对象:

$\text{PolyElem} = \{(c_i, e_i) \mid 1 \leq i \leq n, c_i \in \text{float}, e_i \in \text{int}\};$

数据关系:

$r = \{\langle x_i, y_i \rangle \mid x_i, y_i \in \text{PolyElem}, i = 1, \dots, n-1\}$

基本运算:

初始化和销毁: 分别用于建立空存储结构和释放其空间。

CreateList(fname): 从fname文件中读取数据建立多项式。

Sort(): 对多项式按指数递减排序。

DispPoly(): 输出多项式存储结构。

} //ADT PolyClass

2.5.2 问题求解

1. 设计链式存储结构

- 一个多项式用一个带头结点的单链表存储，每个结点存储一个多项式项 $[c_i, e_i]$ （其中 c_i 为系数， e_i 为指数）

多项式结点类型PolyNode

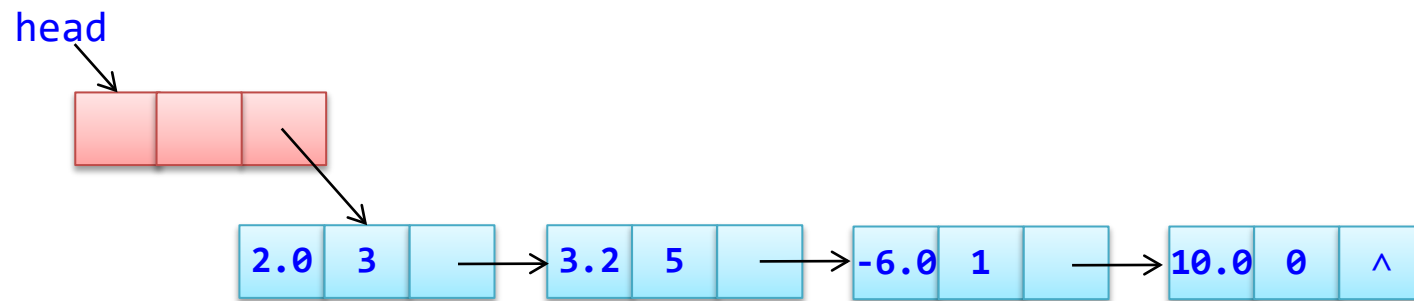
```
struct PolyNode                                //多项式单链表结点类型
{
    double coef;                               //系数
    int exp;                                   //指数
    PolyNode* next;                           //指向下一个结点的指针

    PolyNode():next(NULL) {}                  //构造函数
    PolyNode(double c,int e)                  //重载构造函数
    {
        coef=c;
        exp=e;
        next=NULL;
    }
};
```

- 设计多项式单链表类为PolyList，其中构造函数和析构函数的设计思路与2.3.2节单链表的完全相同。PolyList类的定义如下：

```
class PolyList                                //多项式单链表类
{
public:
    PolyNode* head;                          //多项式单链表的头结点指针
    PolyList()                               //构造函数
    {
        head=new PolyNode();                //建立头结点
    }
    ~PolyList()                              //析构函数
    {   PolyNode* pre=head,*p=pre->next;
        while (p!=NULL)
        {   delete pre;
            pre=p; p=p->next;                //pre、p指针同步后移
        }
        delete pre;
    }
    void CreateList(char* fname);             //读文件采用尾插法建立多项式单链表
    void Sort();                             //对多项式单链表按exp域递减排序
    void DispPoly();                         //输出多项式单链表
};
```

多项式 $p(x)=2x^3+3.2x^5-6x+10$ 对应的单链表:



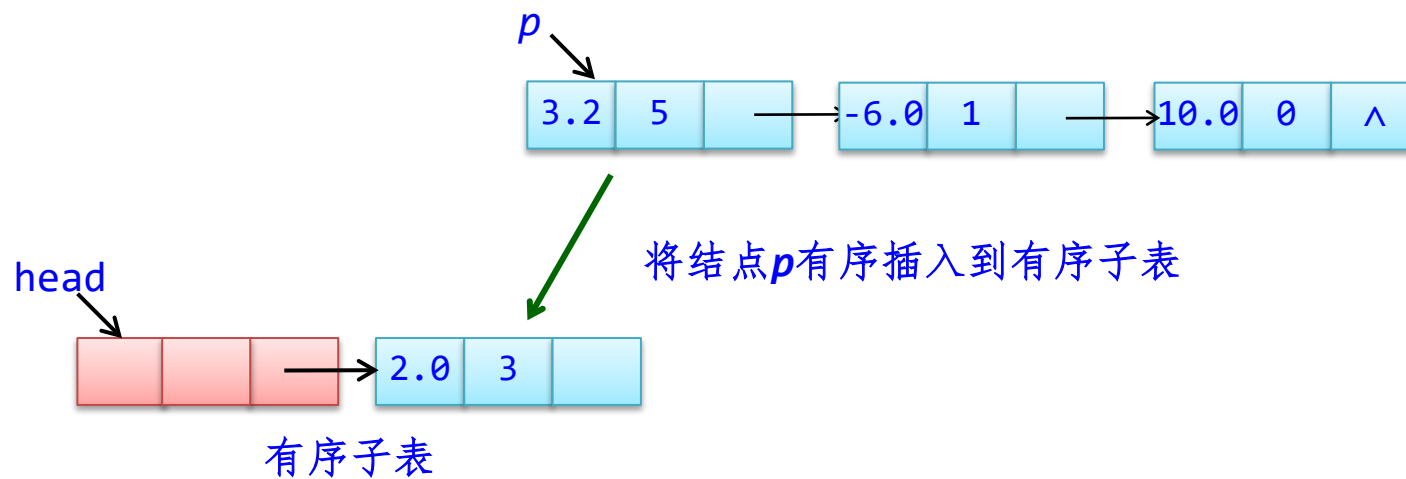
2. 设计PolyList的基本运算算法

(1) 创建多项式单链表CreateList(fname)

```
void CreateList(char* fname) //读文件采用尾插法建立多项式单链表
{
    freopen(fname,"r",stdin); //输入重定向到fname文件
    PolyNode* s,*r;
    double c;
    int n,e;
    scanf("%d",&n);
    r=head; //r始终指向尾结点,开始时指向头结点
    for (int i=0;i<n;i++)
    {
        scanf("%lf%d",&c,&e);
        s=new PolyNode(c,e); //创建新结点s
        r->next=s; //将结点s插入结点r之后
        r=s;
    }
    r->next=NULL; //尾结点next域置为NULL
}
```

读文件+尾插法建表

(2) 多项式单链表排序Sort()



void Sort()	//对多项式单链表按exp域递减排序
{ PolyNode* p,*pre,*q;	
q=head->next;	//q指向开始结点
if (q==NULL) return;	//原单链表空时返回
p=head->next->next;	//p指向结点q的后继结点
if (p==NULL) return;	//原单链表只有一个数据结点时返回
 q->next=NULL;	 //构造只含一个数据结点的有序单链表
while (p!=NULL)	
{ q=p->next;	//q用于临时保存结点p后继结点
pre=head;	//从有序表开头比较
while (pre->next!=NULL && pre->next->exp>p->exp)	
pre=pre->next;	//在有序表中查找插入结点p的前驱结点pre
p->next=pre->next;	//在结点pre之后插入结点p
pre->next=p;	
p=q;	//继续处理原单链表余下的结点
}	
}	

查找到第一个 $\leq p \rightarrow \text{exp}$ 的位置的前驱结点pre

(3) 输出多项式单链表DispPoly()

```
void DispPoly()
{
    bool first=true;
    PolyNode* p=head->next;
    while (p!=NULL)
    {
        if (first)
        {
            printf("[%.11f,%d]",p->coef,p->exp);
            first=false;
        }
        else
            printf(",[%.11f,%d]",p->coef,p->exp);
        p=p->next;
    }
    printf("\n");
}
```

//输出多项式单链表
//first为true表示是第一项
//p指向开始结点

3. 设计两个多项式单链表相加运算算法PolyAdd()

两个按指数递减排序的多项式单链表



二路归并 + 尾插法建表

相加的结果多项式单链表

用pa、pb分别遍历A和B的结点，先建立一个空多项式单链表C，在pa、pb都没有遍历完时循环：

- ① 若pa结点的指数较大，复制pa结点并添加到C的末尾，同时pa后移一个结点。
- ② 若pb结点的指数较大，复制pb结点并添加到C的末尾，同时pb后移一个结点。
- ③ 若pa和pb结点的指数相同，求出它们的系数和c ($c = pa \rightarrow coef + pb \rightarrow coef$)，如果 $c \neq 0$ ，由c和 $pa \rightarrow exp$ 新建一个结点并添加到C的末尾，否则不新建结点，pa、pb均后移一个结点。

上述循环过程结束后，若有一个多项式单链表没有遍历完，说明余下的多项式项都是指数较小的多项式项，将它们均复制并添加到C末尾。

```

void PolyAdd(PolyList& A, PolyList& B, PolyList& C)  //A+B->C
{
    PolyNode* pa=A.head->next;                      //pa指向A的开始结点
    PolyNode* pb=B.head->next;                      //pb指向B的开始结点
    PolyNode* s,*r;
    double c;
    r=C.head;                                        //r指向尾结点

    while (pa!=NULL && pb!=NULL)
    {
        if (pa->exp>pb->exp)                        //归并指数较大的结点pa
        {
            s=new PolyNode(pa->coef,pa->exp);        //复制产生结点s
            r->next=s; r=s;                          //将结点s链到C末尾
            pa=pa->next;
        }
    }
}

```

```

else if (pa->exp<pb->exp)           //归并指数较大的结点pb
{  s=new PolyNode(pb->coef,pb->exp); //复制产生结点s
  r->next=s; r=s;                   //将结点s链到C末尾
  pb=pb->next;
}
else                               //两结点指数相等的情况
{  c=pa->coef+pb->coef;             //求两指数相等结点的系数和c
  if (c!=0)                         //系数和不为0的情况
  {  s=new PolyNode(c,pa->exp);      //新建结点s
    r->next=s; r=s;                 //将结点s链到C末尾
  }
  pa=pa->next;
  pb=pb->next;
}
}

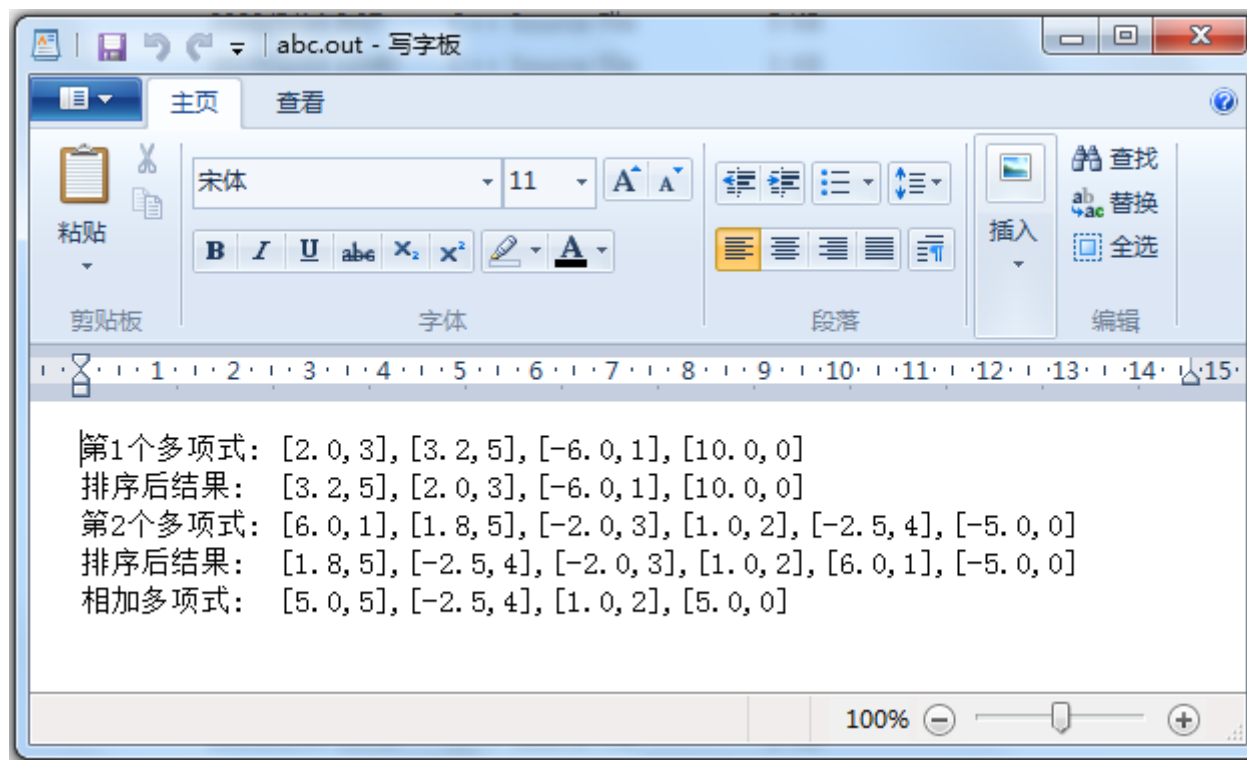
```

```
if (pb!=NULL) pa=pb;           //复制余下的结点
while (pa!=NULL)
{   s=new PolyNode(pa->coef,pa->exp);    //复制产生结点s
    r->next=s; r=s;                    //将结点s链到C末尾
    pa=pa->next;
}
r->next=NULL;                   //尾结点的next域置为NULL
}
```

4. 设计主程序

```
int main()
{   freopen("abc.out","w",stdout);    //输出重定向到abc.out文件
    PolyList A,B,C;                  //建立3个多项式单链表对象
    A.CreateList("abc1.in");
    cout << "第1个多项式: "; A.DispPoly();
    A.Sort();
    cout << "排序后结果: "; A.DispPoly();
    B.CreateList("abc2.in");
    cout << "第2个多项式: "; B.DispPoly();
    B.Sort();
    cout << "排序后结果: "; B.DispPoly();
    PolyAdd(A,B,C);
    cout << "相加多项式: "; C.DispPoly();
    return 0;
}
```

执行程序后打开abc.out文件



2.6 STL中的线性表

- vector-顺序表
- list-链表



自学，上机编程